

# Data Lab 位级运算与浮点表示实验

---

计算机系统基础 Lab1 实验报告

杨悦一 24300220032

2026 年秋季学期

**【摘要】** 本实验在受限的 C 语言运算规则下实现 19 个函数，涉及补码、掩码、字节操作、奇偶校验、整数溢出以及单精度浮点数编码。整数题通过符号掩码、异或折叠和分组处理完成无分支计算，浮点题通过拆分符号位、阶码和有效数实现规格化与最近偶数舍入。终端记录显示，全部函数通过操作符检查，btest 得分为 110/110。本文说明实验要求与符号约定，结合各函数代码分析实现过程和典型输入，并讨论移位边界及类型转换规则方面需要完善的内容。

**【关键词】** 位级运算；补码；掩码；IEEE 754；最近偶数舍入

## 1 引言与实验要求

Data Lab 要求在有限运算符和操作次数内实现指定函数，以加深对机器级数据表示的理解[1]。实验以 bits.c 为核心文件，各题的功能、输入范围、允许运算符及次数上限以源文件说明为依据。

### 1.1 实验任务

实现全部 19 个函数，保持函数接口与题目语义不变。整数题需要处理符号扩展、回绕、比较和饱和边界；浮点题要求直接处理单精度编码，覆盖非规格化数、正负零、NaN、无穷大及舍入进位。

### 1.2 编码与验证要求

整数题只能使用 int 型参数与局部变量，常量范围为 0 至 255，通常仅允许 !、~、&、^、|、+、<<、>>；bitXor 进一步限制为 ~ 和 &。不得使用条件或循环、宏、辅助函数、函数调用、显式类型转换、数组、结构体、联合体及其他数据类型。

浮点题允许 int、unsigned、任意整数常量及整数运算，并允许 if、while 等控制结构；仍不得使用浮点类型或运算、显式类型转换、宏、辅助函数及函数调用。所有题目均需满足各自的操作次数上限。

完成后编译程序，执行 ./check\_ops.py bits.c 与 ./btest，分别检查操作符使用及功能结果。

## 2 符号与操作数介绍

### 2.1 数值表示与符号约定

本文中的  $x$ 、 $y$ 、 $z$ 、 $a$ 、 $b$  通常表示 32 位有符号整数；第 0 位为最低位，第 31 位为最高位。0x 表示十六进制。INT\_MIN= $-2^{31}$ ，其位模式为 0x80000000；INT\_MAX= $2^{31}-1$ ，其位模式为 0x7FFFFFFF。

$n$  表示移位量或幂指数，具体范围依题目而定；src、dst 表示源与目标字节下标。mask 表示用于筛选位的掩码； $q$ 、 $r$  分别表示商和余数，half 表示半个舍入单位。正文中的减号、乘号和比较符号用于数学解释，不代表整数题可以直接使用这些 C 运算符。

uf 表示以 unsigned 保存的单精度浮点编码：sign 为第 31 位，exp 为第 30 至 23 位的 8 位阶码，frac 为低 23 位尾数。规格化数的无偏指数为  $\text{exp}-127$ ，隐含有效位为 1；非规格化数不含该隐含 1。sig 表示恢复后的有效数，rem 表示移位舍弃的余数。

### 2.2 运算符含义

整数题的基本运算符见表 1。按位运算针对每一位独立处理；逻辑非只判断整个操作数是否为零。特别地， $\wedge$  在 C 中表示异或，而不是乘方。

表 1 整数题基本运算符

| 符号 | 含义   | 说明或示例                    |
|----|------|--------------------------|
| !  | 逻辑非  | !0=1，非零值取逻辑非为 0          |
| ~  | 按位取反 | 每个 0 变为 1，每个 1 变为 0      |
| &  | 按位与  | 两位都为 1 时结果为 1；用于掩码       |
|    | 按位或  | 至少一位为 1 时结果为 1；用于合并      |
| ^  | 按位异或 | 两位不同时为 1； $x \wedge x=0$ |
| +  | 加法   | 本实验按 32 位模型保留结果低 32 位    |
| << | 左移   | 低位补 0；合法移位量为 0 至 31      |
| >> | 右移   | int 算术右移；unsigned 逻辑右移   |

### 2.3 操作数类型与操作次数

操作数是运算符作用的对象，可以是允许范围内的常量、函数参数或局部变量。例如  $x \& 0xFF$  的操作数为  $x$  和 0xFF； $\sim x$  只有一个操作数。较大的掩码需要由小常量移位、按位或等方式构造，不能在整数题中直接写成超过 255 的常量。

操作数个数与操作次数不同： $\sim x + 1$  含一次取反和一次加法，共两次操作； $(x \gg 31) \& (\sim x + 1)$  共四次。赋值不计入操作次数。浮点题可用 unsigned，右移时高位补零；代码中的后缀 u 表示无符号整型常量。

## 3 实验环境与验证过程

### 3.1 编译环境与命令

实验在 Windows 下的 WSL Ubuntu 终端中进行，使用 GCC 与 Make。图 1 记录先清理再编译的过程；命令采用 `-O -Wall -fwrapv -m32` 选项，生成 `btest`、`fshow` 和 `ishow`。分析遵循题目给定的 32 位补码、算术右移及保留低 32 位的位向量模型。

```
• yyy@yyy:~/test-git/ICS-26Fall-FDU-DataLab$ make clean
rm -f *.o btest fshow ishow *~
• yyy@yyy:~/test-git/ICS-26Fall-FDU-DataLab$ make all
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
```

图 1 make clean 与 make all 的终端记录

### 3.2 操作符检查

执行 `./check_ops.py bits.c`。图 2 列出每题使用次数与上限，并显示 19 个函数全部通过操作符检查。操作次数和上限已在截图中给出。

```
• yyy@yyy:~/test-git/ICS-26Fall-FDU-DataLab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 16/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 19/24 operators
midpointTowardFirst: 29/32 operators
isBetweenEitherOrder: 45/48 operators
mul5Sat: 25/30 operators
classifyAdd3: 31/52 operators
floatScaleThreeHalves: 34/60 operators
floatRoundEven: 25/65 operators
float_i2f: 37/40 operators
bitCount: 40/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图 2 ./check\_ops.py bits.c 的终端输出

### 3.3 功能测试

执行 `./btest`。图 3 中所有函数的 Errors 均为 0，总分为 110/110。该结果说明该次测试通过；函数的全部输入边界与编码规则仍需结合源代码审查。

```
● yyy@yyy:~/test-git/ICS-26Fall-FDU-DataLab$ ./btest
Score  Rating  Errors  Function
  1      1      0    signMask
  2      2      0    bitXor
  3      3      0   negativePart
  4      4      0   copyByteWithin
  4      4      0   logicalShift
  4      4      0   swapNibblePairs
  4      4      0 secondLowestZeroBit
  5      5      0    oddParity
  5      5      0 rotateRightBits
  5      5      0 roundEvenPow2
  5      5      0 midpointTowardFirst
  7      7      0 isBetweenEitherOrder
  7      7      0    mul5Sat
  7      7      0 classifyAdd3
  7      7      0 floatScaleThreeHalves
 10     10      0 floatRoundEven
 10     10      0 float_i2f
 10     10      0    bitCount
 10     10      0    bitReverse
Total points: 110/110
```

图 3 `./btest` 的终端输出

## 4 函数实现与分析

以下代码对应 `bits.c` 中的 19 个函数，仅整理缩进、空行及过长行的换行。每题依次说明功能要求、解题思路、具体实现和示例；示例用于解释位运算过程，不作为额外执行测试的记录。

### 4.1 signMask

题目要求：返回仅最高有效位为 1 的 32 位掩码 `0x80000000`。

思路：利用移位把最低位的 1 直接送到符号位。

```
int signMask(void) {
    return 1<<31;
}
```

具体实现：将常数 1 左移 31 位，得到仅最高位为 1 的 `0x80000000`。在本实验的 32 位位向量模型下，该位模式同时表示 `INT_MIN`。表达式 `1<<31` 只需要一次操作。

示例：输出为 `0x80000000`，二进制形式是一个 1 后接 31 个 0；将其与 `0x87654321` 按位与，仅保留最高位。

## 4.2 bitXor

题目要求：仅使用按位取反  $\sim$  与按位与  $\&$ ，实现  $x$  与  $y$  的按位异或。

思路：从异或真值条件出发，排除两位同为 0 或同为 1 的情况。

```
int bitXor(int x, int y) {  
    return ~(x&y)&~(~x&~y);  
}
```

具体实现：异或要求对应位恰好有一个为 1。实现式  $\sim(x \& y) \& \sim(\sim x \& \sim y)$  同时排除“两位均为 1”和“两位均为 0”的情况，因此只保留不同位。该式通过德摩根律构造异或，仅使用题目允许的  $\sim$  和  $\&$ 。

示例： $x=4$  (0100)、 $y=5$  (0101) 时，仅最低位不同，返回 1； $x=y=7$  时对应位完全一致，返回 0。

## 4.3 negativePart

题目要求：当  $x < 0$  时返回补码取负结果，否则返回 0；结果按 32 位模型解释。

思路：将符号位扩展为全位掩码，选择取负结果或零，避免条件分支。

```
int negativePart(int x){  
    return (x>>31)&(~x+1);  
}
```

具体实现：利用算术右移， $x \gg 31$  在  $x$  为负时生成全 1 掩码，否则生成全 0 掩码。 $\sim x + 1$  为补码取负，将两者按位与即可在负数时返回取负结果，在非负数时返回 0。对  $\text{INT\_MIN}$ ，其相反数超出有符号范围，结果按实验模型保留为  $0x80000000$ ，而非可表示的正整数。

示例： $x=-10$  时， $x \gg 31$  为全 1， $\sim x + 1$  为 10，返回 10； $x=5$  时掩码全 0，返回 0。 $\text{INT\_MIN}$  的取负位模式仍为  $0x80000000$ 。

## 4.4 copyByteWithin

题目要求：将  $x$  的第  $\text{src}$  个字节复制到第  $\text{dst}$  个字节，其他字节不变；字节从低到高编号 0 至 3。

思路：按“提取源字节、清空目标位置、写回源字节”分三步完成复制。

```
int copyByteWithin(int x, int src, int dst) {  
    int shift = dst << 3;  
    int byte = (x >> (src << 3)) & 0xFF;  
    int mask = 0xFF << shift;  
    return (x & ~mask) | (byte << shift);  
}
```

具体实现：先将源字节下标  $\text{src}$  左移 3 位得到比特偏移量，通过右移和  $\& 0xFF$  提取源字节。再构造目标掩码  $0xFF \ll (\text{dst} \ll 3)$ ，用  $x \& \sim \text{mask}$  清空目标字节，最后将源字节移到目标位置并按位或。源字节来自原始  $x$ ，因此  $\text{src}$  与  $\text{dst}$  相等时也能保持原值，其他字节不会改变。

示例： $x=0x11223344$ 、 $\text{src}=0$ 、 $\text{dst}=2$  时，提取  $0x44$ ，清空后得到  $0x11003344$ ，再写入得到  $0x11443344$ 。

## 4.5 logicalShift

题目要求：对  $x$  实现逻辑右移  $n$  位，高位补 0； $0 \leq n \leq 31$ 。

思路：先算术右移，再用低位掩码清除符号扩展位。

```
int logicalShift(int x, int n) {
    return (x >> n) & ~(((1 << 31) >> n) << 1);
}
```

具体实现：先执行算术右移  $x \gg n$ ，再清除符号扩展产生的高位。表达式  $\sim(((1 \ll 31) \gg n) \ll 1)$  构造低  $32-n$  位为 1 的掩码，与右移结果按位与后即得到逻辑右移。 $n=0$  时掩码为全 1， $n=31$  时掩码为 1，两个边界均不需要移位 32 位。

示例： $x=0x87654321$ 、 $n=4$  时，算术右移得到  $0xF8765432$ ；与  $0xFFFFFFFF$  相与后得到  $0x08765432$ 。

## 4.6 swapNibblePairs

题目要求：交换  $x$  的每个字节内部的高 4 位与低 4 位。

思路：分别提取两个半字节，向相反方向移动 4 位后合并。

```
int swapNibblePairs(int x) {
    int maskF0 = (240 << 24) | (240 << 16) | (240 << 8) | 240;
    int mask0F = ~maskF0;
    return ((x & mask0F) << 4) | (((x & maskF0) >> 4) & ~(~0xF << 24));
}
```

具体实现：用常数 240 在四个字节中构造  $0xF0F0F0F0$ ，其按位取反得到  $0x0F0F0F0F$ 。每个字节的低半字节取出后左移 4 位，高半字节取出后右移 4 位，最后按位或完成交换。由于右移是算术右移，最高字节的高半字节还需用  $0xFFFFFFFF$  掩码清除符号扩展位，该掩码由  $\sim(\sim 0xF \ll 24)$  构造。

示例：字节  $0xAB$  的低半字节  $B$  左移到高端，高半字节  $A$  右移到低端，得到  $0xBA$ ；对  $0x12345678$ ，结果为  $0x21436587$ 。

## 4.7 secondLowestZeroBit

题目要求：返回标记从低到高第二个 0 位的掩码；不足两个 0 位时返回 0。

思路：将找 0 位转化为找 1 位，先清除最低 1，再隔离新的最低 1。

```
int secondLowestZeroBit(int x) {
    int a = ~x;
    int b = a & (a + (~0));
    return b ^ (b & (b + (~0)));
}
```

具体实现：令  $a = \sim x$ ，将原数中的 0 位转换为 1 位。恒等式  $a \& (a-1)$  能清除最低的 1，因此  $b = a \& (a + \sim 0)$  保留第二个及更高的 1。随后计算  $b \wedge (b \& (b-1))$ ，仅留下  $b$  的最低 1，也就是原数第二个最低 0 所在位置。如果原数不足两个 0 位，则  $b=0$ ，返回值自然为 0。

示例： $x=0xFFFFFFFF$  时， $a=0x5$ ，清除最低 1 后  $b=0x4$ ，返回  $0x4$ ； $x=0x7FFFFFFF$  只有一个 0 位，返回 0。

## 4.8 oddParity

题目要求：返回奇校验位：x 中 1 的个数为偶数时返回 1，为奇数时返回 0。

思路：异或可以表示奇偶性，通过分治折叠把全部位的奇偶性集中到最低位。

```
int oddParity(int x) {
    x = x ^ (x >> 16);
    x = x ^ (x >> 8);
    x = x ^ (x >> 4);
    x = x ^ (x >> 2);
    x = x ^ (x >> 1);
    return !(x & 1);
}
```

具体实现：按 16、8、4、2、1 的步长，将 x 与其右移结果反复异或，使所有 32 位的异或结果最终汇聚到最低位。最低位为 1 表示原数中 1 的个数为奇数，为 0 表示偶数。题目要求返回补足奇校验的校验位，因此使用  $!(x \& 1)$ ，在原数含偶数个 1 时返回 1。

示例：5 的二进制为 0101，含两个 1，需补 1 才形成奇校验，返回 1；7 为 0111，含三个 1，返回 0。

## 4.9 rotateRightBits

题目要求：将 32 位 x 循环右移 n 位，移出的低位补回高端；题目注释给出  $n \geq 0$ 。

思路：把循环右移拆成逻辑右移与左移回填两部分。

```
int rotateRightBits(int x, int n) {
    int k = n & 31;
    int left = (32 + (~k + 1)) & 31;
    int mask = ~(((1 << 31) >> k) << 1);
    int right = (x >> k) & mask;
    return right | (x << left);
}
```

具体实现：实现由两部分组成：第一部分以与 logicalShift 相同的掩码得到 x 的逻辑右移 n 位结果；第二部分将 x 左移 32-n 位，把被移出的低 n 位放回高端，两部分按位或得到循环右移。示例：x=0x12345678、n=8 时，逻辑右移部分为 0x00123456，回填部分为 0x78000000，按位或后得到 0x78123456。

## 4.10 roundEvenPow2

题目要求：将非负 x 舍入为最接近的  $2^n$  的倍数，中点选择商为偶数的倍数； $0 \leq x \leq 0x3ffffff$ ， $1 \leq n \leq 16$ 。

思路：比较余数与半个单位，处于中点时用商的最低位决定是否进位。

```
int roundEvenPow2(int x, int n) {
    int mask = (1 << n) + ~0;
    int half = 1 << (n + ~0);
    int q = x >> n;
    int r = x & mask;
    int greater = !((r + ~half) >> 31);
    int equal = !(r ^ half);
    int up = greater | (equal & (q & 1));
    return (q + up) << n;
}
```

具体实现：将  $x$  分解为商  $q=x \gg n$  和余数  $r=x \& ((1 \ll n)-1)$ ，令  $half=2^{(n-1)}$ 。余数大于  $half$  时进位，余数等于  $half$  时仅在  $q$  为奇数时进位，使结果商为偶数。代码用  $r + \sim half$ ，即  $r-half-1$  的符号判断严格大于关系，再用  $!(r \wedge half)$  判断相等，最终返回  $(q + up) \ll n$ 。

示例： $n=2$  时，10 的商为 2、余数为 2，正好处于中点且商为偶数，返回 8；14 的商为 3，需进位，返回 16。

#### 4.11 midpointTowardFirst

题目要求：无溢出地计算数学中点  $(x+y)/2$ ；若中点位于两个整数之间，则选择更接近第一个参数  $x$  的整数。

思路：先构造不会溢出的向下取整中点，再依据两数顺序决定是否加 1。

```
int midpointTowardFirst(int x, int y) {
    int mid = (x & y) + ((x ^ y) >> 1);
    int odd = (x ^ y) & 1;
    int sx = (x >> 31) & 1;
    int sy = (y >> 31) & 1;
    int diffSign = ((x + (~y + 1)) >> 31) & 1;
    int greater = ((sx ^ sy) & (!sx)) | (!(sx ^ sy) & !diffSign & !(x ^ y));
    return mid + (odd & greater);
}
```

具体实现：先用  $(x \& y) + ((x \wedge y) \gg 1)$  计算向下取整的中点，避免直接求  $x+y$  导致溢出。最低位  $(x \wedge y) \& 1$  表示和是否为奇数。若和为奇数且  $x > y$ ，则在向下取整的中点上加 1，使结果靠近第一个参数；其余情况保持原值。

判断  $x > y$  时，异号情况直接根据符号确定，同号情况才使用  $x-y$  的符号并排除  $x=y$ ，避免把异号相减的溢出结果当作比较依据。例如 (4,7) 返回 5，交换参数为 (7,4) 后返回 6；(INT\_MIN, INT\_MAX) 的结果分别随参数顺序取 -1 或 0。

示例： $(x,y)=(4,7)$  时， $mid=5$ 、 $odd=1$ ，但  $x < y$ ，不加 1，返回 5；参数交换后  $x > y$ ，返回 6。

#### 4.12 isBetweenEitherOrder

题目要求：判断  $x$  是否在以  $a$ 、 $b$  为端点的闭区间内；端点顺序任意，满足时返回 1，否则返回 0。

思路：直接合并两种端点顺序下的区间条件，并用符号位保护大小比较。

```

int isBetweenEitherOrder(int x, int a, int b) {
    int sx = (x >> 31) & 1;
    int sa = (a >> 31) & 1;
    int sb = (b >> 31) & 1;
    int ax = sa ^ sx;
    int bx = sb ^ sx;
    int xa = ((x + (~a + 1)) >> 31) & 1;
    int xb = ((x + (~b + 1)) >> 31) & 1;
    int a_le_x = (ax & sa) | (!ax & !xa);
    int x_le_a = (ax & sx) | ((!ax) & (xa | !(x ^ a)));
    int b_le_x = (bx & sb) | (!bx & !xb);
    int x_le_b = (bx & sx) | ((!bx) & (xb | !(x ^ b)));
    return (a_le_x & x_le_b) | (b_le_x & x_le_a);
}

```

具体实现：区间条件可写为  $(a \leq x \text{ 且 } x \leq b)$  或  $(b \leq x \text{ 且 } x \leq a)$ 。代码分别构造四个比较结果，不必先对端点排序。比较  $x$  与  $a$  时，若两者异号，直接由符号判断大小；若同号，利用  $x-a$  的符号，并在需要时用  $!(x \wedge a)$  包含相等情况。与  $b$  的比较同理。

这种分支掩码方式既能处理  $a > b$ ，也能处理  $x$  等于任一端点以及  $a = b$  的退化区间。异号时不采用减法结果的符号，从而避免跨越 `INT_MIN` 和 `INT_MAX` 的比较出现溢出误判。

示例： $x=5$ 、 $a=8$ 、 $b=3$  时，第二组条件  $b \leq x \leq a$  成立，返回 1； $x=9$  时两组条件均不成立，返回 0； $x=3$  时包含端点，返回 1。

#### 4.13 mul5Sat

题目要求：计算  $x \times 5$ ；超出 `int` 正向范围时返回 `INT_MAX`，超出负向范围时返回 `INT_MIN`。

思路：先判断幅值是否超过安全阈值，再用全位掩码选择普通乘法结果或饱和值。

```

int mul5Sat(int x) {
    int sign = x >> 31;
    int ax = (x ^ sign) + (sign & 1);
    int limit = (0x19 << 24) |
                (0x99 << 16) |
                (0x99 << 8) | 0x99;
    int overflow = (ax >> 31) |
                  ~((ax + ~limit) >> 31);
    int tmin = 1 << 31;
    int sat = tmin ^ ~sign;
    int y = (x << 2) + x;
    return (overflow & sat) | (~overflow & y);
}

```

具体实现：先用  $\text{sign} = x \gg 31$  和  $(x \wedge \text{sign}) + (\text{sign} \& 1)$  构造绝对值的位模式。再以小常数组组合出阈值 `0x19999999`，即 429496729。正负两侧可安全乘 5 的最大绝对值均为该值；超过阈值时应饱和，而不是返回回绕结果。

$ax + \sim\text{limit}$  等于  $ax - \text{limit} - 1$ ，用其符号构造“绝对值超过阈值”的全位掩码；`INT_MIN` 的绝对值位模式仍为负数，则由  $ax \gg 31$  单独识别。正常结果以  $(x \ll 2) + x$  计算。饱和值  $\text{tmin} \wedge \sim\text{sign}$  对非负输入得到 `INT_MAX`，对负输入得到 `INT_MIN`，最终按溢出掩码选择两者。

示例： $x=429496729$  时， $5x=2147483645$ ，仍可表示； $x=429496730$  时， $5x$  超过 `INT_MAX`，返回 2147483647；负向越界则返回 -2147483648。

#### 4.14 classifyAdd3

题目要求：判断精确数学和  $x+y+z$  是否越界：高于 `INT_MAX` 返回 1，低于 `INT_MIN` 返回 -1，否则返回 0；不得使用更宽整数类型。

思路：累计两次加法正向、负向溢出修正量，让相互抵消的溢出得到正确归类。

```
int classifyAdd3(int x, int y, int z) {
    int s = x + y;
    int t = s + z;
    int sx = (x >> 31) & 1;
    int sy = (y >> 31) & 1;
    int sz = (z >> 31) & 1;
    int ss = (s >> 31) & 1;
    int st = (t >> 31) & 1;
    int pos1 = !sx & !sy & ss;
    int neg1 = sx & sy & !ss;
    int pos2 = !ss & !sz & st;
    int neg2 = ss & sz & !st;
    int pos = pos1 + pos2;
    int neg = neg1 + neg2;
    return pos + (~neg + 1);
}
```

具体实现：先计算两次回绕加法  $s=x+y$ 、 $t=s+z$ ，并分别判断每次加法是否产生正向或负向溢出。两个非负数相加得到负数表示正向溢出，两个负数相加得到非负数表示负向溢出。代码分别累计 `pos` 和 `neg`，返回 `pos-neg`。

其依据是：每次正向溢出使精确和比回绕结果多  $2^{32}$ ，每次负向溢出使其少  $2^{32}$ ，因此精确总和可表示为  $t + (\text{pos} - \text{neg}) \cdot 2^{32}$ 。三个 32 位有符号整数相加的范围限制了净修正量只能为 -1、0、1。净修正量为 0 时  $t$  就是可表示的精确和；为正或负时分别超过上界或低于下界。

示例：`INT_MAX+1-1` 的中间加法先正溢出，随后负溢出，两次抵消，返回 0；`INT_MAX+INT_MAX+0` 返回 1；`INT_MIN+INT_MIN+0` 返回 -1。

## 4.15 floatScaleThreeHalves

题目要求：对 `uf` 表示的单精度浮点数计算  $f \times 3/2$ ，使用最近偶数舍入；保留正负零，NaN 原样返回，结果仍为浮点编码。

思路：在整数域中对有效数乘 3，再调整指数并统一进行最近偶数舍入。

```
unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exp = (uf >> 23) & 0xFFu;
    unsigned frac = uf & 0x7FFFFFFu;
    if (exp == 255)
        return uf;
    if (exp == 0 && frac == 0)
        return uf;
    unsigned sig = frac;
    unsigned e = exp;
    if (exp != 0) {
        sig |= 1u << 23;
    } else {
        e = 1;
    }
    unsigned p = sig * 3u;
    unsigned shift = 1;
    if (p >= (1u << 25)) {
        shift = 2;
        e++;
    }
    unsigned q = p >> shift;
    unsigned mask = (1u << shift) - 1;
    unsigned rem = p & mask;
    unsigned half = 1u << (shift - 1);
    if (rem > half || (rem == half && (q & 1)))
        q++;
    if (q >= (1u << 24)) {
        q >>= 1;
        e++;
    }
    if (exp == 0 && q < (1u << 23))
        e = 0;
    if (e >= 255)
        return sign | 0x7F800000u;
    return sign | (e << 23) | (q & 0x7FFFFFFu);
}
```

具体实现：将 `uf` 拆分为 `sign`、`exp` 和 `frac`。`exp=255` 时原样返回，以保留 NaN 和无穷大；正负零也原样返回。对规格化数补上隐含最高位得到有效数 `sig`；对非规格化数不补隐含位，并暂以 `e=1` 统一指数尺度。随后计算整数乘积  $p=3 \cdot \text{sig}$ 。

通常将 `p` 右移 1 位实现乘  $3/2$ ；若  $p \geq 2^{25}$ ，则右移 2 位并将指数加 1，以维持有效数的规格化范围。根据被舍弃的余数 `rem` 与 `half` 的大小执行最近偶数舍入：大于中点时加 1，等于中点时仅在保留结果 `q` 的最低位为 1 时加 1。该方法直接对精确的整数乘积舍入一次。

若舍入后 `q` 达到  $2^{24}$ ，则再次右移并增加指数。原非规格化数的 `q` 若仍小于  $2^{23}$ ，则将指数设回 0；否则转为规格化数。指数达到 255 时返回相应符号的无穷大，最后重新组合符号、指数与尾数。

示例：`uf=0x3F800000` 表示 1.0，结果为 `0x3FC00000`，即 1.5。最小正非规格化数 `uf=1` 乘 1.5 后位于编码 1 与 2 的中点，按偶数规则取编码 2。

## 4.16 floatRoundEven

题目要求：将 `uf` 表示的浮点数舍入到最近整数，中点取偶数，并返回该整数的浮点编码；保留零的符号，NaN 与无穷大原样返回。

思路：按指数把输入分成小于半单位、位于半单位附近、含小数位及已为整数的几类。

```
unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exp = (uf >> 23) & 0xFFu;
    unsigned frac = uf & 0x7FFFFFFu;
    if (exp == 255)
        return uf;
    if (exp < 126)
        return sign;
    if (exp == 126) {
        if (frac == 0)
            return sign;
        return sign | (127u << 23);
    }
    int e = exp - 127;
    if (e >= 23)
        return uf;
    unsigned sig = (1u << 23) | frac;
    unsigned shift = 23 - e;
    unsigned mask = (1u << shift) - 1;
    unsigned rem = sig & mask;
    unsigned half = 1u << (shift - 1);
    unsigned integer = sig >> shift;
    if (rem > half ||
        (rem == half && (integer & 1))) {
        integer++;
    }
    unsigned result = integer << shift;
    if (result >= (1u << 24)) {
        exp++;
        result >>= 1;
    }
    return sign | (exp << 23) |
        (result & 0x7FFFFFFu);
}
```

具体实现：目标是将浮点值舍入到最近的偶数规则整数，再返回该整数的浮点编码。首先对 `exp=255` 的输入原样返回。`exp<126` 时绝对值小于 0.5，直接返回带原符号的零；`exp=126` 时，尾数为 0 表示恰好  $\pm 0.5$ ，应舍入到  $\pm 0$ ，尾数非零则舍入到  $\pm 1$ 。

对其余有限数，令 `e=exp-127`。`e $\geq$ 23` 时数值已没有小数位，直接返回；否则补上隐含位，按 `shift=23-e` 将有效数分为整数部分与余数。余数大于半单位时进位，等于半单位且整数部分为奇数时进位。最后恢复有效数位置，并处理进位导致的指数增加。负数沿用幅值舍入后恢复符号，因此能保留舍入到零时的负号。

示例：2.5 的保留整数部分为 2，正好中点且为偶数，舍入为 2.0；3.5 的保留整数部分为奇数，进位到 4.0；-0.5 舍入为 -0，编码为 0x80000000。

## 4.17 float\_i2f

题目要求：将 int 型 x 转换为单精度浮点数的位级编码，以 unsigned 返回；不得实际使用浮点类型或浮点运算。

思路：把绝对值规格化，提取最高有效位后的 23 位，再根据舍弃位进行舍入。

```
unsigned float_i2f(int x) {
    unsigned sign = 0;
    unsigned abs_x = 0;
    unsigned temp;
    unsigned exp;
    unsigned frac;
    unsigned mask;
    unsigned round_bits;
    sign = (x >> 31) & (1<<31);
    if (x==0) return 0;
    if (x>0||x<0) abs_x = (x ^ (x >> 31)) + (~ (x >> 31)+1);
    temp = abs_x;
    exp = 127 + 31;
    mask = 1 << 31;
    while (((temp & mask) >> 31) == 0) && temp) {
        temp <<= 1;
        exp+=~0;
    }
    frac = temp << 1 >> 9;
    round_bits = temp << 24;
    if (round_bits > (1<<31)) {
        frac+=1;
    }
    if (round_bits == (1<<31)) {
        if (frac & 1) frac+=1;
    }
    if (frac >> 23) {
        frac = 0;
        exp+=1;
    }
    return sign | (exp << 23) | frac;
}
```

具体实现：先提取符号位；x=0 时返回零编码。对非零 x，利用符号掩码构造无符号绝对值。INT\_MIN 的幅值按 32 位模型保留为 0x80000000，存入 unsigned 后可正确表示。随后不断左移 temp，直到最高有效 1 到达第 31 位；指数从 127+31 开始，每左移一次减 1。

表达式 temp << 1 >> 9 去掉最高有效 1 并保留后续 23 位尾数；temp << 24 将被舍弃的低 8 位移到高端，与 0x80000000 比较以判断是否超过一半。恰好等于一半时，仅在尾数最低位为 1 时进位。若尾数进位溢出 23 位，则清零尾数并将指数加 1，最后与符号位合并。

示例：x=5 的二进制为 101，规格化为  $1.01 \times 2^2$ ，指数为 129，尾数为 0x200000，得到 0x40A00000；16777217 处于两个可表示浮点数的中点，舍入为 16777216。

## 4.18 bitCount

题目要求：统计 32 位  $x$  的二进制表示中 1 的个数，返回值范围为 0 至 32。

思路：将每位的 0 或 1 视为初始计数，在越来越大的分组内并行相加。

```
int bitCount(int x) {
    int m1_1 = (85<<8)|85;
    int m1 = (m1_1<<16)|m1_1;
    int m2_1 = 51;
    int m2 = (m2_1 << 24) | (m2_1 << 16) | (m2_1 << 8) | m2_1;
    int m4_1 = 15;
    int m4 = (m4_1 << 24) | (m4_1 << 16) | (m4_1 << 8) | m4_1;
    int m8_1 = 255;
    int m8 = (m8_1 << 16) | m8_1;
    int m16 = (m8_1 << 8) | m8_1;
    x = (x & m1) + ((x >> 1) & m1);
    x = (x & m2) + ((x >> 2) & m2);
    x = (x & m4) + ((x >> 4) & m4);
    x = (x & m8) + ((x >> 8) & m8);
    x = (x & m16) + ((x >> 16) & m16);
    return x;
}
```

具体实现：采用分组并行计数。先构造 0x55555555、0x33333333、0x0F0F0F0F、0x00FF00FF 和 0x0000FFFF 掩码，分别按 1、2、4、8、16 位分组累加。每一步都将低半组与右移后的高半组相加，使组内保存该组的 1 的个数。最终 32 位汇总为一个 0 至 32 的整数。右移结果与掩码相与后，符号扩展不会影响计数。

示例： $x=7$  (0111) 时，第一层把相邻位计数为 2 和 1，后续合并得到 3； $x=-1$  的 32 位全为 1，最终返回 32。

## 4.19 bitReverse

题目要求：反转 32 位  $x$  的所有二进制位，使原第  $i$  位移动到第  $31-i$  位。

思路：按 1、2、4、8、16 位的尺度逐级交换相邻半区。

```
int bitReverse(int x)
{
    int m16=(255<<8)|255;
    int m8=m16^(m16<<8);
    int m4=m8^(m8<<4);
    int m2=m4^(m4<<2);
    int m1=m2^(m2<<1);
    x=((x & m1)<< 1)|((x >> 1) & m1);
    x=((x & m2)<< 2)|((x >> 2) & m2);
    x=((x & m4)<< 4)|((x >> 4) & m4);
    x=((x & m8)<< 8)|((x >> 8) & m8);
    x=(x<<16)|((x>>16)&m16);
    return x;
}
```

具体实现：与分组计数相似，按 1、2、4、8、16 位逐级交换每组的两个半区。掩码从 0x0000FFFF 开始，通过与自身左移结果异或，依次生成 0x00FF00FF、0x0F0F0F0F、0x33333333 和 0x55555555，减少构造成本。前四步将掩码保留的低半区左移、高半区右移后合并，最后交换高低 16 位。所有位经历五级交换后，原第  $i$  位移到第  $31-i$  位。

示例：0x80000004 的第 31 位和第 2 位为 1，反转后变为第 0 位和第 29 位，结果为 0x20000001。

## 5 结论

实验通过 19 个函数将数值表示与位级运算联系起来。符号掩码用于无分支选择，异或折叠用于校验，分组处理用于计数与反转，商余数分解及最低保留位共同决定最近偶数舍入。已有终端结果为 110/110；后续应完善循环移位边界和类型转换规则，使功能正确性与编码规范得到一致验证。

## 参考文献

[1] 复旦大学计算机系统基础课程. Lab 1 Data Lab 实验页面[EB/OL]. <https://ics-26fall-fdu.github.io/labs/lab1-data-lab/>.